

Execution- and Prediction-Based Auto-Tuning of Parallel Read and Write Parameters

Megha Agarwal[†], Pragya Jain[‡], Divyansh Singhvi[§], Preeti Malakar[¶]

Indian Institute of Technology Kanpur

[†]meghaagr@iitk.ac.in, [‡]pjjain@iitk.ac.in, [§]dsinghvi@iitk.ac.in, [¶]pmalakar@iitk.ac.in

Abstract—Parallel I/O tuning is useful for scientific applications that read and write huge amounts of data. I/O performance depends on multiple tunable parameters such as the stripe size, stripe count, the collective I/O buffer size, and the number of collective I/O aggregators. The search space being large, it is cumbersome to tune the I/O parameters for every system to achieve optimal results. We propose active learning-based execution and prediction-based tuning models. These recommend a good set of I/O parameter values for an application on a given system. These models use optimization to find the parameter values; the objective is to minimize I/O time. The models allow to focus on improvement of read and/or write performance, and separate tuning of reads and writes. We evaluated our models using I/O kernels of scientific applications (S3D-IO, BT-IO and GenericIO) and the highly configurable IOR benchmark on an Intel-based supercomputer, HPC2010. We achieved an increase in I/O bandwidth of up to $8x$ over the default parameters, when both read and write are optimized together, and up to $20x$ in read bandwidths when optimized separately.

Index Terms—Parallel I/O, machine learning, active learning, performance prediction, tuning

I. INTRODUCTION

Large-scale scientific simulations output enormous amount of data [26] and may exhibit high read/write times. One of the reasons is disparity in the I/O and network bandwidth and the compute speeds. Parallel I/O in high performance computing environments implies input/output data movement from the compute nodes to the parallel file system (storage). Often the I/O data is forwarded through the I/O forwarding nodes [7] to better optimize the reads and writes to the storage. The MPI library provides several tunable parallel I/O knobs that may improve the performance. For example, collective buffer size and number of aggregator nodes are some of the important parameters in case of MPI collective I/O. These may affect the amount of data movement among processes, and between the MPI processes and the I/O forwarding nodes, or the storage servers. Parallel file systems such as Lustre also provides various tunable parameters that can help realize higher I/O bandwidths.

The parameters affect the overall I/O time [17]. For example, larger stripe size and stripe count in the Lustre file system may reduce I/O times depending on the data size. Larger collective buffer size may reduce the I/O times depending on the network bandwidth. However, the challenge is that the optimal parameters depend on the number of nodes, data size and other factors [13] such as the I/O paths, file

system, network and application data access patterns [11], [12]. It is challenging for a user to determine the optimal parameter values in order to obtain the best I/O performance. I/O parameter tuning also require a detailed understanding of the data flows involved during I/O which may be unknown to the users. Therefore application developers often use the default parameters. However, default parameters may not give the best performance for all applications. Therefore it is essential to set these optimally.

Automatically tuning the I/O parameters has been studied in the past [9], [11] using genetic algorithms, heuristic search etc. This helps avoid multiple test runs to empirically determine satisfactory I/O parameters; however some of these may take a few hours [8], [12]. Our approach automatically tunes the I/O parameters and suggests a good set of parameters within a few minutes. In our earlier work [6], we explored tuning both reads and writes together. In this work, we propose separate tuning models for read and write, because the same parameters may not optimize both. We proposed two auto-tuning models based on active learning, wherein the model itself selects the samples for training. We used Bayesian optimization to select the parameters based on the previous runs, thereby reducing the search space. The active learning model builds a probability model to recommend the tunable parameters. This model is based on previous evaluations of the objective function (I/O bandwidth in our case). The user only needs to select a range of each tunable I/O parameter. We have considered various MPI I/O and Lustre parameters.

We provide the option to build a combined model for I/O, as well as separate auto-tuning models for reads and writes. This is useful when an application is read- or write-oriented. Our first model, an execution-based auto-tuning model, uses the actual execution times to search for better parameters in less than 20 iterations. Our second model, a prediction-based auto-tuning model, uses predicted values of execution times to search the parameter space. We observed that for some applications, the maximum improvement with a combined tuning model was only $1.3\times$, whereas separate tuning models for reads and writes gave $20\times$ maximum improvement. The combined model to simultaneously optimize both read and write operations may not give the most desirable set of parameter values. This is because the combined model tries to optimize both read and write jointly. We train separate

models for read and write such that the models are able to independently find good parameters for reads and writes in case of read- or write-heavy applications.

We showed improvements in I/O performance on the HPC2010 supercomputer at the Indian Institute of Technology Kanpur, India. We tuned the parameters – stripe size, stripe count, romio_ds_read, romio_ds_write, cb_nodes, romio_cb_read, romio_cb_write and cb_buffer_size. We used the IOR benchmark and a few scientific application I/O kernels. We achieved a maximum improvement of $8\times$ and $20\times$ using the combined and separate read-write models respectively, as compared to the bandwidth obtained with the default parameters. In summary, we developed

- 1) active learning-based auto-tuning approach that improves I/O performance.
- 2) an execution-based tuning of read and write parameters, that gives up to $8\times$ improvement.
- 3) a fast prediction-based tuning that can tune I/O parameters in less than a minute.
- 4) a model to tune read and write parameters separately, that is able to achieve up to $20\times$ improvement.

The remaining sections are organized as follows. We compare and contrast with related work in §II. Our method is explained in §III and results are shown in §IV, followed by summary in §V.

II. RELATED WORK

Many prior works have proposed auto-tuning of I/O parameters using genetic algorithms, Bayesian optimization and gradient descent [8], [9], [11]–[13], [27]. Behzad et al. uses genetic algorithm to search the I/O parameters. However, this incurs a long run time and is inapplicable on unseen configurations, whereas, our prediction-based auto-tuning model is able to predict for unseen configurations and then tune the parameters. Bagbaba et al. [8] propose genetic algorithm based auto-tuning of I/O parameters. However, their approach takes hours. McLay et al. studied the effects of various combinations of parallel I/O parameters, mainly stripe size and count [30]. They also experimented with multiple writers in collective I/O. However, manual tuning being tedious and time-consuming, the users generally resort to the default parameters. In contrast, we propose a method that recommends a good set of I/O parameters within seconds.

Barker et al. [10] and Song et al. [34] proposed analytical models to predict the I/O cost. Analytical models for predicting the I/O parameters is a challenging problem, given the growing complexity of network topology, hardware and parallel I/O system software. Therefore, machine learning models have been explored. Behzad et al. use non-linear regression models [11] and semi-empirical approaches [12] for performance prediction of I/O. Herbein et al. use a statistical model to predict the performance of the I/O operations [29]. Isaila et al. [22] model the performance of ROMIO collectives using a combination of analytical and ML approaches. Xie et al. identify important features and

used linear regression to model I/O [37]. In contrast, we use ML to predict read and write times separately. Further, we use these predictions to lower the execution time of our models to tune the read and write parameters separately.

III. PARALLEL I/O PARAMETER TUNING

Parallel I/O data flow rate may be optimized using correct I/O parameterizations depending on the components that are involved in I/O such as the compute node architecture and node intraconnect of multicore nodes [28], the network connectivity between the compute nodes and the I/O forwarding nodes, and the network speed. It is challenging to achieve the best results (maximum I/O speed) because there are many tunable factors that affect the I/O rates. These include the MPI I/O parameters such as the number of aggregators for collective MPI I/O [14], the collective I/O buffer size, whether or not to perform data sieving and file system parameters such as the stripe size and stripe count in case of Lustre [25]. The best parameter values change with data sizes even for the same application, thus adding to the complexity of optimal parameter selection.

The I/O parameters need to be tuned based on the application, data size, file access patterns and resources. This huge search space of input parameters makes it challenging to optimally tune them. For example, stripe size for an application using Lustre may be set between 1 MB – 16 MB depending on the file system block size (most supercomputing centers set it to 1 MB by default) and stripe count may be set up to the maximum number of object storage targets (OST) (≥ 200 [18] in some cases). Thus, the search space with these two parameters is > 3000 if the stripe size is varied in units of 1 MB. The search space increases manifold if we also include the MPI-IO parameters. We address this challenge by automatically tuning the I/O parameters using machine learning (ML). Specifically, we used active learning [33] to reduce the search space so that it is feasible to tune within a few minutes. Next, we describe our active learning based auto-tuning model that uses real execution runs for tuning in Section III-A, I/O bandwidth prediction model and prediction-based auto-tuning model in Section III-B.

A. Execution-based Auto-tuning (ExAct)

A simple approach to obtain the best parameters is to execute the application using all possible parameter values on a given system [35], [36]. However, this is infeasible, and it incurs a large computation overhead due to the huge parameter space. Active learning helps achieve good results with fewer training set data [33]. This is possible by interactively querying the information source to obtain the output for a new training data point. This is especially applicable in scenarios like ours where obtaining training data is an expensive process (considering the required compute time to conduct empirical experiments for generating training data). The active learning model selects the training data points, so that it can learn faster and perform better with less

training data. For this, the active learning algorithm queries those points where the error is higher.

We used Bayesian Optimization (BO) [23], [31] to find the optimal values of MPI-IO hints and Lustre [25] parameters (see §IV-A) that give the highest I/O bandwidth (best performance) for a given data size of an application on a given system. Search space of each parameter may be a set of values in case of discrete space and a range for parameters that belong to continuous space. We used uniform function as the probability function associated with the search space because the probability of finding optimal parameters anywhere in the entire range is same. We state the problem as $x^* = \arg\min_{x \in \mathcal{X}} f(x)$, where the set of values for tunable parameters is denoted by x^* . The goal is to minimize the objective function $f(x)$ which is reciprocal of I/O bandwidth in our case. $\mathcal{X}$ denotes the parameter space. BO builds a probability model of the objective function. BO keeps track of previous evaluations by computing $P(\text{objective function score} | \text{parameters})$. This model is called as surrogate for objective function. It is constructed using Bayes rule, $p(y|x) = \frac{p(x|y)p(y)}{p(x)}$, where $p(x|y)$ is the probability of the set of parameters given score.

We provide input configurations such as the problem size and initial MPI I/O and Lustre parameters to the algorithm. BO takes these parameters as input and outputs the loss function. The next set of parameters are selected based on those that gave good results in the previous runs [1], [19]. With these parameters, the application code is run (automatically) to obtain I/O bandwidth. The output (i.e. the current performance) is used to calculate the loss and update the surrogate model. The model tries to minimize the loss (i.e. inverse of I/O bandwidth) and selects the next set of parameters based on that. We used Expected Improvement (EI) criteria to select the next set of parameters. $EI_{y^*}(x) = \int_{-\infty}^{y^*} (y^* - y)p(y|x)dy$, where, y^* is a threshold value of the objective function and the actual value of the objective function is y . The objective is to maximize expected improvement with respect to x . These parameter values are used in the next run through environment variables. This avoids source code modifications. The three steps (parameter selection, running the code and updating model) are done for user-defined number of steps (we ran for 20 iterations because it was sufficient to converge). The model takes insignificant time to select parameters for the next run in comparison to the execution time of the application at each step in order to update the surrogate model. BO tries to converge in fewer iterations than other search methods such as random search. Every time the algorithm finds the next set of candidate values, these are evaluated with the actual objective function. This result (score, parameter values) is recorded and they form the history which are used in subsequent runs.

We experimented with a combined model and also a separate model for read and write. When an application is both read and write intensive, the users may use one objective function that may yield the best parameters for both read and

write because the application both reads and writes. However, for those applications that are either read-intensive or write-intensive, we built different models for reads and writes. In case of separate ExAct models for read and write, the objective functions are read and write performance (inverse of bandwidth) respectively. Some of the MPI-IO parameters and Lustre parameters are different for reads and writes. For example, the MPI-IO hints *romio_ds_read* and *cb_read* are specifically used for parallel reads, while *romio_ds_write* and *cb_write* are used for parallel writes. Therefore, we constructed separate models for read and write. In this case, the parameter space is slightly reduced because read parameters are not applicable in case of writes, and vice versa. After the parameter selection step, the application only performs reads in case of read model, and performs only writes in case of write model. Based on the output performance, the surrogate model is updated.

The model runtime is proportional to the product of the number of iterations and the application execution time. Bayesian optimization converges faster because it takes less time to find a feasible solution. However, the total time may be large depending on the application runtime. Therefore, we used a performance prediction model to predict the I/O bandwidth instead of running the application. We use separate prediction models for reads and writes to update the separate surrogate models. We describe our faster prediction-based active learning model next.

B. Prediction-based Auto-tuning (PrAct)

We used machine learning algorithms to predict I/O performance, so that the performance values (predicted I/O bandwidths) can be used to update the surrogate model. We used extreme gradient boosting (XGB) [16], which is known to work well for non-linear relationships between the input parameters [27]. The input data consists of the number of collective I/O aggregator nodes, collective buffer size, problem size, Lustre file system stripe size and count, the number of nodes. The model output is read/write bandwidth. The input data may be collected from the previous runs that the user may have run. We used the data from the ExAct runs, prior run job logs can be also used to feed the input data to XGB model. Since we also use separate active learning models for read and write, we built separate read/write prediction models as well. Implementation details are presented in Section IV-A.

Next, we used the output of the prediction model to improve the performance of the ExAct model, we denote this prediction-based model as PrAct. It is able to learn the MPI I/O and Lustre parameters in a few seconds because PrAct does not need to run the application, it uses the predicted values (combined or separate for reads and writes) to update the surrogate model (combined or separate for reads and writes). This reduces the execution time by more than 40× than the execution-based model. We trained separate read/write PrAct models as well as combined model. The separate model is used for applications that are either read

or write-intensive. In this case, the combined model tries to optimize both read and write operations simultaneously which may not be able to learn the optimal parameter values.

IV. EXPERIMENTS AND RESULTS

We now describe the implementation details, the benchmarks and the evaluation systems. We next demonstrate the improvement in I/O performance using our model.

A. Experiment Details

We used GenericIO [20] which is the I/O kernel for the HACC cosmology simulation code. The input are seed and the number of particles. We experimented with terabytes of data using this benchmark. We used S3D-IO [32] which is an I/O kernel for S3D [15]. We used the non-blocking collective I/O option using the PnetCDF library. The inputs are the 3D grid sizes and number of processes in each dimension. We used BT-IO [24] that is an I/O kernel for the Block Tri-diagonal solver (BT) code of NAS Parallel Benchmarks, which can be used to test PnetCDF and MPI-IO performance for the I/O pattern of BT. The inputs are the 3D grid sizes. We used PnetCDF non-blocking I/O in case of GenericIO and BT-IO. We also used the Interleaved or Random I/O benchmark (IOR) [4], [21], which is a highly configurable I/O benchmark developed at Lawrence Livermore National Laboratory (LLNL). We experimented with various transfer and block sizes. We used the MPIIO mode. We used “-U” and “-O” options to set the MPI-IO hints and Lustre file system options respectively.

The active learning models are implemented in Python using the Hyperopt [3] library for hyperparameter optimization. We applied 0-1 normalization to both input and output before training and inverse transform after prediction. The optimization algorithm used in Hyperopt is Tree-structured Parzen estimator (TPE). It initializes each parameter on the basis of bias defined on search space. This was uniform distribution between the range of values in our case. We use environment variables to set the I/O hints which can be automatically set during the ExAct runs. We tuned some of the important parameters such as Lustre `stripe_size`, `stripe_count`, `cb_buffer_size`, `cb_nodes`, `romio_ds_read/write`, and `romio_cb_read/write`. Our model can be used to tune any application without any modifications. The application developer simply needs to modify the execution command line of the application in our scripts and the job script depending on the system where the job will run.

We used 1000 trees and maximum tree depth of 10 for the XGB prediction model. We had around 400 – 1000 training data points per application. We did a 10-fold cross validation with 30:70 training:test splits. In the combined prediction model, we have a single performance prediction model which predicts the combined runtime (read time + write time) of an application, for given values of Lustre and MPI-IO parameters. In the separate prediction model, there are separate prediction models for read and write which predict the read and write bandwidths (or time) respectively

of an application, for given values of parameters. The Predict model is used as objective function in PrAct. The results are obtained on new configurations. The Predict and PrAct models (both combined and separate) were tested on two I/O benchmarks – S3D-IO and BT-IO. The results were averaged over 5 runs, both for default and model-selected values.

Training time for ExAct is the number of iterations we run the application multiplied by the execution time. However, we run less than 5 time steps for each benchmark. The PrAct training time is ~ 11 seconds (data-dependent). Thus, it takes ~ 1000 s on average to obtain best parameters from ExAct for GenericIO, while it takes only ~ 18 s for PrAct ($\sim 40\times$ reduction). We ran our experiments using a maximum of 224 processes on the supercomputer HPC2010 [2] at the Indian Institute of Technology Kanpur. This system comprises of 372 Intel Xeon (Nehalem) nodes, connected by QDR Infiniband. Each node has 8 cores and 48 GB RAM. The network topology is fat-tree. The nodes are connected to the Lustre file system (v2.5, 100 TB storage) with 24 OSTs. The default stripe size is 1M and the default stripe count is 1 on this system. Many larger installations consists of higher number of OSTs, however, this only affects the range of search space of OSTs. Thus the performance of our models would still be better than random search.

B. Results

The experiments have been repeated 5 times, and the error bars have been shown for all results to capture the variability, that is typical of I/O times.

1) *Execution-based Auto-tuning (ExAct)*: The results for GenericIO combined model were obtained on 16, 32, 128 and 224 processes of HPC2010. The number of particles for the runs ranges from 26 million – 1 billion particles. We obtained an average improvement of $1.8\times$ and $2.1\times$ for reads and writes, and a maximum improvement of $3.6\times$ and $3.9\times$ across all runs. The read bandwidth ranged from about 2 GBps for 16 processes (2 nodes) to 9 GBps for 224 processes (28 nodes) for 1 billion particles, in comparison to 1 GBps – 5 GBps in case of default parameters. This is because ExAct model selects better parameters to optimize the I/O times. For example, for one of the runs on 28 nodes with 1 billion particles, ExAct recommended a `stripe_count` of 16 whereas the default is 1, and a `stripe_size` of 128 MB whereas the default is 1 MB. Since 1 billion particle runs generated TBs of data, changing the default file system parameters optimally led to better performance. We show later the parameter differences between ExAct and default that led to higher I/O bandwidths.

We tested BT-IO on 2 and 8 nodes, with 8 processes per node on the HPC2010 supercomputer. We achieved an average improvement of $1.2\times$ and $2.6\times$, and maximum improvements up to $1.4\times$ and $4.6\times$ for read and write bandwidths respectively. We achieved almost similar average and maximum improvement with combined model. Thus our model is capable of recommending optimal parameters for applications that are both read and write intensive by

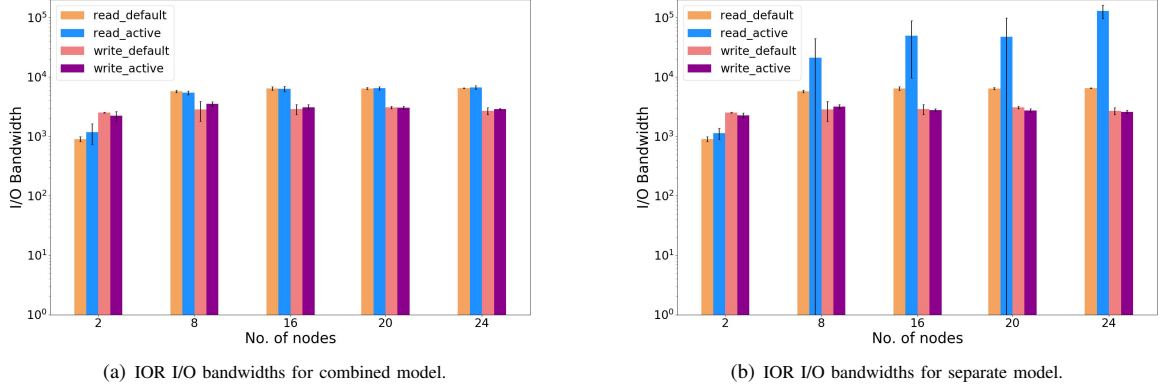


Fig. 1: Default vs. ExAct I/O bandwidths (MBps) for read-write ExAct model using IOR on HPC2010. Strong scaling on 16 – 256 processes (a) Combined model (b) Separate model, log scale on Y-axis.

minimizing the loss function. Since the I/O pattern in BT-IO is not contiguous (due to the block tri-diagonal matrix layout), it is extremely important to optimally tune the writes. As the results demonstrate, the separate model is able to suggest better parameters for larger writes and reads (see $400 \times 400 \times 400$ and $500 \times 500 \times 500$).

Figure 1 shows strong scaling of IOR on HPC2010 for combined and separate read-write ExAct models. We tested the read and write performance separately using the configurable parameters of IOR. The combined model results in only $1.3\times$ maximum improvement for read bandwidth over the read bandwidth obtained using the default parameters, whereas the separate model for read results in $8\times$ average improvement and $20\times$ maximum improvement upon using the parameters recommended by the ExAct model for read. The reason for this is because the separate read-write ExAct models try to minimize read and write performance separately and thus we are able to get good read performance using these tunings. The writes generate more traffic and contention, and thus may require more iterations to find the optimal parameters. We got about $1.2\times$ improvement in writes in both cases. The search space is huge, yet our model is useful in determining the read and write parameters by minimizing the loss function and converges in a few iterations.

We ran S3D-IO on a maximum of 128 processes with grid sizes between $100 \times 100 \times 100$ to $400 \times 400 \times 400$, with unequal x-y-z extents as well. We achieved a maximum of $2.5\times$ and $7.5\times$ improvement in read and write bandwidths over the default across 40 different configurations. This gain is due to optimal parameter selection by ExAct. For e.g., our model recommended parameters yielded a write bandwidth of 2.15 GBps, whereas the average default bandwidth was 0.32 GBps for the configuration $400 \times 400 \times 400$ on 64 processes. This was due to using `stripe_count=16` as compared to default of 1. Also, ExAct gives more performance improvements for the same data size at higher process counts. This is because it auto-tunes useful parameters such as the number of OSTs, which leads to higher concurrency and thus higher I/O bandwidths. The I/O time for an input grid

size of $200 \times 400 \times 400$ on 128 processes was 37.5 seconds (averaged over 5 runs) using the default parameters. With ExAct parameters, the I/O time was reduced significantly. For e.g., one of the runs resulted in an I/O time of 6.7 seconds. This was because the model enabled aggregation, and used `stripe_count=17` (default=1), and used a larger `cb_buffer_size` of 256 MB (default=4 MB).

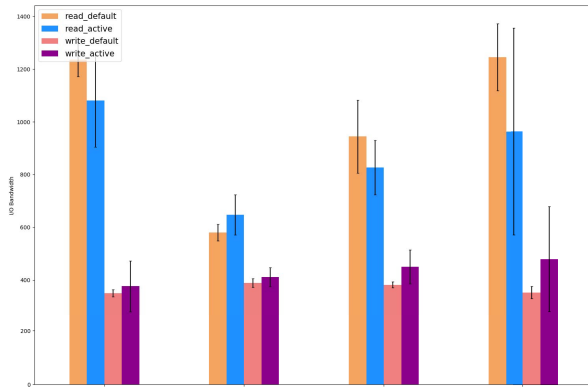
Table I shows parameters chosen by separate read-write ExAct model (last 8 columns), the resulting bandwidths (columns 3 and 4), and the default parameters and corresponding bandwidths (bottom sub-row of every row) for a few configurations for each of the benchmarks (column 1). For e.g., in case of S3D-IO run of $100 \times 400 \times 400$ on 16 nodes, ExAct selects `stripe_size` of 8 over the default value of 1, a higher `cb_buffer_size` of 512 MB over default value of 1 MB, disables data-sieving and enables collective I/O optimisation. The first two parameters (columns 5 and 6) pertain to the file system and the remaining to the MPI I/O layer. The parameters from our ExAct model results in an improvement of up to $3.4\times$ and $6\times$ in read and write bandwidths respectively. The table shows that the default parameters may not be the optimal parameters in most cases.

2) *Prediction-based Auto-tuning (PrAct)*: Now, we show the results from PrAct model which uses our XGB-based performance prediction model as the I/O bandwidth estimate instead of actually running the application. For each input configuration, PrAct runs 100 iterations to converge. We show the results from both combined and separate PrAct models (which use combined and separate prediction models respectively). Combined PrAct model gives us the I/O parameters that optimise the combined I/O (read + write) time. Separate PrAct model optimizes the I/O parameters separately using separate read and write prediction models. We show the PrAct model results for S3D-IO and BT-IO in Figures 2 and 3 respectively. The blue bars show the PrAct read bandwidths, the orange bars show the default read bandwidths. The baseline results are from the runs with default parameters. It can be noted that the separate model for PrAct has clearly benefited the BO model. There is no

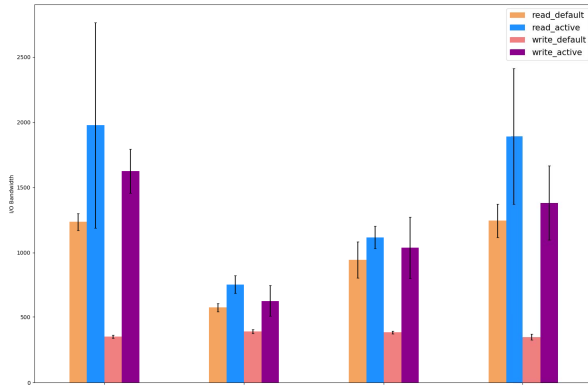
Benchmark	Configuration	Bandwidth (MBps)		Parameters (ExAct (top sub-row) Default (bottom sub-row))							
		Read	Write	ss (MB)	sc	ds_read	ds_write	cb_n	cb_read	cb_write	cb_bf (MB)
S3D-IO	Grid size: $100 \times 400 \times 400$, Nodes: 16	3155.32	1598.33	2.88	8	disable	disable	12	enable	enable	512
		942.02	265.47	1	1	automatic	automatic	16	automatic	automatic	4
IOR	Transfer size: 4M, Block size: 100M, Nodes: 24	7308	2886.53	16.75	6	enable	disable	12	enable	disable	16
		6378	2327.09	1	1	automatic	automatic	24	automatic	automatic	4
BT-IO	Grid size: $400 \times 400 \times 400$, Nodes: 8	1606.43	1068.95	6.44	5	disable	disable	7	enable	enable	1024
		835.96	290.02	1	1	automatic	automatic	8	automatic	automatic	4
GenericIO	Data size: 512M, Nodes: 16	3320.51	7944.88	31	16	disable	enable	15	disable	disable	128
		684.779	2496.69	1	1	automatic	automatic	16	automatic	automatic	4

TABLE I: Separate read-write ExAct auto-tuning model on HPC2010. The hints are, in order, *stripe_size* (ss), *stripe_count* (sc), *romio_ds_read* (ds_read), *romio_ds_write* (ds_write), *cb_nodes* (cb_n), *romio_cb_read* (cb_read), *romio_cb_write* (cb_write) and *cb_buffer_size* (cb_bf). The two rows for a particular benchmark and configuration indicate the bandwidths achieved by using the parameter values obtained by ExAct model (ExAct) and the default parameters (Default) respectively.

advantage of combined model for some of the configurations in both the benchmarks. This may be partly due to combined prediction model.



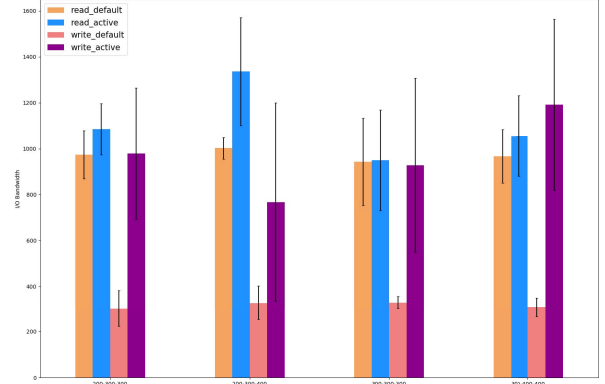
(a) S3D-IO: Combined PrAct model.



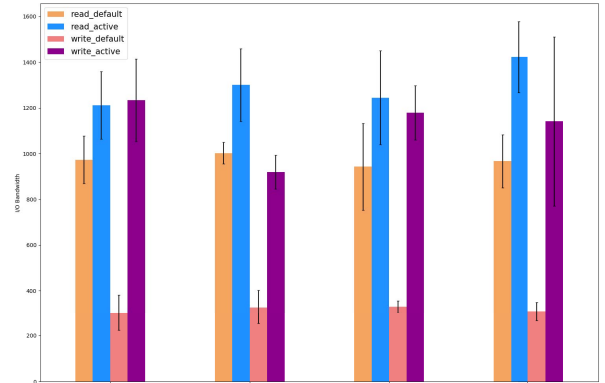
(b) S3D-IO: Separate PrAct model.

Fig. 2: (a) Combined and (b) separate PrAct model results for S3D-IO on 2 – 16 nodes of HPC2010. Bandwidths shown on Y-axis (MBps).

We ran S3D-IO and BT-IO over various new configurations. We observe that PrAct is able to perform well even for these new configurations with the predictions. The average read and write improvements for the combined model for S3D-IO were $0.91\times$ and $1.17\times$ respectively. The average read and write improvements for the separate model for S3D-IO were $1.40\times$ and $3.23\times$ respectively. The average read and write improvements for the combined model for BT-IO were $1.14\times$ and $3.07\times$ respectively. The average read and write improvements for the separate model for BT-IO



(a) BT-IO: Combined PrAct model.



(b) BT-IO: Separate PrAct model.

Fig. 3: (a) Combined and (b) separate PrAct model results for varying 3D grid sizes of input data (X-axis) for BT-IO. Bandwidths shown on Y-axis (MBps).

were $1.33\times$ and $3.54\times$ respectively. Improvements in write bandwidths are higher than for reads. This may be due to less lock contention owing to optimized data flows to the storage. Separate PrAct model shows an improvement of up to 32% over combined model. This is because the same set of parameter values may not be optimal for both reads and writes. Separate read-write model provides us the flexibility to tune different set of I/O parameters to optimise read and write independently. We ran the experiments for a few time steps. However, our models can help lower time to read/write data from parallel shared file systems by a few hours for long-running production runs.

Table II shows the parameters chosen by the separate

Benchmark	Configuration	Bandwidth (MBps)		Parameters								
		PrAct(r)	PrAct(w)	Default								
		Read	Write	ss (MB)	sc	ds_read	ds_write	cb_n	cb_read	cb_write	cb_bf (MB)	
S3D-IO	Grid size: $300 \times 300 \times 480$ Nodes: 16	2836.62	991.16	1	16	disable	disable	15	disable	disable	8	
		2371.85	1871.16	27.75	14	disable	disable	10	disable	enable	256	
		1170.57	353.45	1	1	automatic	automatic	16	automatic	automatic	4	
BT-IO	Grid size: $300 \times 400 \times 480$ Nodes: 8	1591.16	40.73	31.19	4	enable	disable	7	enable	disable	1024	
		277.17	1579.45	3.5	13	disable	disable	8	disable	enable	1024	
		997.92	277.94	1	1	automatic	automatic	8	automatic	automatic	4	

TABLE II: PrAct (separate read-write) auto-tuning model on HPC2010. The hints are *stripe_size* (ss), *stripe_count* (sc), *romio_ds_read* (ds_read), *romio_ds_write* (ds_write), *cb_nodes* (cb_n), *romio_cb_read* (cb_read), *romio_cb_write* (cb_write), *cb_buffer_size* (cb_bf). The three rows for a particular benchmark and configuration indicate the bandwidths achieved by using the parameter values predicted by separate read PrAct Model (PrAct(r)), separate write PrAct Model (PrAct(w)), and the default parameters (Default) respectively.

read-write PrAct model (last 8 columns), the resulting bandwidths (columns 3 and 4) and the default parameters and corresponding bandwidths for a few configurations of S3D-IO and BT-IO benchmarks. The sub-rows show the parameters for separate read PrAct model (PrAct(r)), separate write PrAct model (PrAct(w)) and the default parameters. Note that the separate read PrAct model focus on maximising read bandwidths, and that the tuned parameters may not give good write bandwidths. Similarly the separate write PrAct model focus on maximizing the write bandwidths. For e.g., the separate PrAct models for BT-IO (second row) selects *stripe_size* = 4 for read and *stripe_size* = 13 for write over the default value of 1. As a result, the read and write models result in 1.6 $\times$ and 5.7 $\times$ improvements in read and write bandwidth respectively.

The training time of PrAct is less than 20 seconds on average, whereas the average time to train ExAct model is 800 seconds which is less than time taken by existing models. The training time of PrAct is lower than ExAct because the PrAct model uses the predictions whereas ExAct uses the real runtimes of the application.

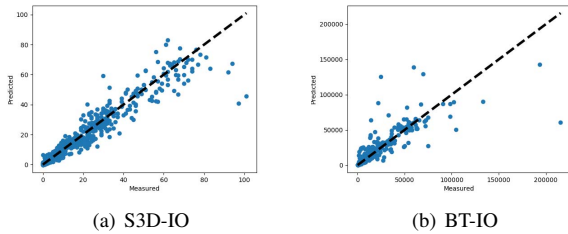


Fig. 4: Scatter plots of values predicted by XGB vs. measured values of total (read+write) time on combined Predict models for S3D-IO and BT-IO on HPC2010 for 30/70 split of train/test data. Ideal graph is the dashed black line; blue dots show actual values.

The PrAct results also show the utility of our performance prediction model. We now show the prediction model results. The training and test data for our extreme gradient boosting-based performance prediction model was generated from the ExAct runs. The production run log data can also be used to train the XGB model. We ran XGB [5] regression model to predict the I/O bandwidths. We trained both combined and separate read-write prediction models for S3D-IO and BT-IO, using the data obtained from combined and separate ExAct models respectively. Separate prediction models involve two separate performance prediction models for read and write bandwidths. Figures 4 and 5 show the performance of XGB predictions (combined and separate read-write prediction

model respectively) for S3D-IO and BT-IO on HPC2010. Note that all the plots are centered around the black dotted line which is the ideal scenario of 0% prediction error. These results are average of 10-fold cross validations in 30 : 70 train:test ratio. Note that typical ML models train on 80% data. However, we trained with only 30% data and yet get good results. We noted higher accuracy with 50% training data (not shown here). However, with our hyperparameter tuning, XGB gave decent results even with 30% train data (as seen from the figures), that could be used in PrAct model.

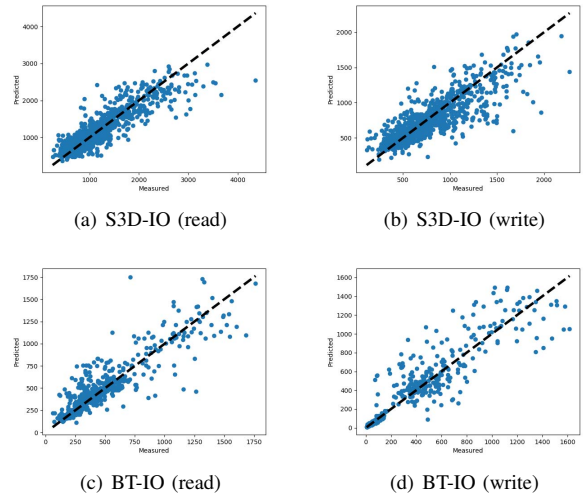


Fig. 5: Scatter plots of values predicted by XGB vs. measured values of read ((a), (c)) and write ((b), (d)) bandwidths on separate read-write Predict models for S3D-IO and BT-IO on HPC2010 for 30% training and 70% test data. The dashed black line is the ideal graph; actual values are denoted by blue dots.

V. CONCLUSIONS

In this work, we proposed two models based on active learning. Both use Bayesian optimization for minimizing the loss function in order to search the parameter space faster. One of the models, which is execution-based (ExAct), uses the real application run times to update the surrogate model. The second model, which is prediction-based (PrAct), uses the predicted values of I/O bandwidths to update the surrogate model. The model outputs optimal I/O parameters (MPI-IO and Lustre parameters). The search range of the parameters (example the number of OSTs, the collective buffer size range) and the distribution (uniform in our case) is provided to the BO model. Our models can be quickly trained for a generic I/O kernel/benchmark. We obtained

up to $20\times$ improvements in I/O bandwidths on up to 224 cores on HPC2010. Across all benchmarks, we achieved improvements between $1.2\times - 5\times$ for reads and writes using the combined model. Separate ExAct model significantly outperformed the combined model by achieving an average improvement of $8\times$ and maximum improvement of up to $20\times$ (over default values) for writes in IOR. We conclude that, in most of the cases, the hints may be needed to be tuned separately to achieve both optimal reads and writes, especially for applications that are either read- or write-intensive.

We also presented an XGB-based performance model to predict the read and write bandwidths separately. This is used by the PrAct model. PrAct approach of active learning reduces the training time to a few seconds or minutes. The time taken for the 100 prediction iterations is negligible. So, the runtime of combined PrAct model is $20\times$ improvement over ExAct model, which runs every configuration for 20 iterations. Separate PrAct model takes approximately double the time of combined PrAct model due to separate modeling. Nevertheless, it is still an improvement over the modeling time using ExAct. PrAct resulted in improvements of up to $1.6\times$ and $4.7\times$ in read and write bandwidths respectively. For both, ExAct and PrAct, write bandwidths were better tuned (higher improvements) over reads. The separate read/write models were successful in tuning parameters such that improvements specifically in read or write could be prioritized, and showed an improvement of up to 32% over the combined PrAct model. We demonstrated that auto-tuning I/O parameters may benefit from active learning. In future, we plan to experiment further with the separate read/write models, better prediction models and test them on more benchmarks.

REFERENCES

- [1] Automated Hyperparameters tuning. <https://www.kaggle.com/willkoehrsen/automated-model-tuning>.
- [2] HPC2010. <https://www.hpc.iitk.ac.in>.
- [3] Hyperopt. <https://github.com/hyperopt/hyperopt/wiki/FMin>.
- [4] IOR Code. <https://github.com/hpc/ior>.
- [5] Xgboost. <https://xgboost.readthedocs.io/en/latest/>.
- [6] M. Agarwal, D. Singhvi, P. Malakar, and S. Byna. Active Learning-based Automatic Tuning and Prediction of Parallel I/O Performance. In *2019 IEEE/ACM Fourth International Parallel Data Systems Workshop (PDSW)*, pages 20–29, 2019.
- [7] N. Ali, P. Carns, K. Iskra, D. Kimpe, S. Lang, R. Latham, R. Ross, L. Ward, and P. Sadayappan. Scalable I/O forwarding framework for high-performance computing systems. In *2009 IEEE International Conference on Cluster Computing and Workshops*, 2009.
- [8] Aye Bagbaba and Xuan Wang. Improving the MPI-IO Performance of Applications with Genetic Algorithm based Auto-tuning. In *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2021.
- [9] P. Balaprakash, J. Dongarra, T. Gambin, M. Hall, J. K. Hollingsworth, B. Norris, and R. Vuduc. Autotuning in High-Performance Computing Applications. *Proceedings of the IEEE*, 106(11), 2018.
- [10] K. J. Barker, K. Davis, and D. J. Kerbyson. Performance modeling in action: Performance prediction of a Cray XT4 system during upgrade. In *International Symposium on Parallel Distributed Processing*, 2009.
- [11] Babak Behzad, Surendra Byna, Prabhat, and Marc Snir. Optimizing I/O performance of HPC applications with autotuning. *Transactions on Parallel Computing*, 5(4):15:1–15:27, 2019.
- [12] Babak Behzad, Surendra Byna, Stefan M Wild, Mr Prabhat, and Marc Snir. Improving parallel I/O autotuning with performance modeling. In *Proceedings of the 23rd International Symposium on High-performance Parallel and Distributed Computing*, pages 253–256, 2014.
- [13] Babak Behzad et al. Taming parallel I/O complexity with auto-tuning. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, page 68. ACM, 2013.
- [14] M. Chaarawi and E. Gabriel. Automatically Selecting the Number of Aggregators for Collective I/O Operations. In *2011 IEEE International Conference on Cluster Computing*, pages 428–437, 2011.
- [15] Jacqueline H Chen and et al. Terascale direct numerical simulations of turbulent combustion using S3D. *Computational Science & Discovery*, 2(1), 2009.
- [16] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. *CoRR*, abs/1603.02754, 2016.
- [17] G. Congiu, S. Narasimhamurthy, T. Sub, and A. Brinkmann. Improving Collective I/O Performance Using Non-volatile Memory Devices. In *IEEE International Conference on Cluster Computing*, 2016.
- [18] Cori. <http://www.nersc.gov/users/computational-systems/cori>.
- [19] Peter I. Frazier. A Tutorial on Bayesian Optimization. *arXiv e-prints*, Jul 2018.
- [20] GenericIO. <https://git.cels.anl.gov/hacc/genericio>.
- [21] IOR. <https://buildmedia.readthedocs.org/media/pdf/ior/latest/ior.pdf>.
- [22] Florin Isaila, Prasanna Balaprakash, Stefan M Wild, Dries Kimpe, Rob Latham, Rob Ross, and Paul Hovland. Collective I/O tuning using analytical and machine learning models. In *IEEE International Conference on Cluster Computing*, 2015.
- [23] Patrick W. Koch, Brett Wujek, Oleg Golovidov, and Steven Gardner. Automated Hyperparameter Tuning for Effective Machine Learning. In *Proceedings of the SAS Global Forum 2017 Conference*, 2017.
- [24] Wei-keng Liao. Design and evaluation of MPI file domain partitioning methods under extent-based file locking protocol. *IEEE Transactions on Parallel and Distributed Systems*, 22(2):260–272, 2011.
- [25] Lustre. <http://www.lustre.org/>.
- [26] Huong Luu et al. A Multiplatform Study of I/O Behavior on Petascale Supercomputers. In *Proceedings of the International Symposium on High-Performance Parallel and Distributed Computing*, pages 33–44, 2015.
- [27] P. Malakar and et al. Benchmarking Machine Learning Methods for Performance Modeling of Scientific Applications. In *Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems*, pages 33–44, 2018.
- [28] P. Malakar and et al. Topology-aware space-shared co-analysis of large-scale molecular dynamics simulations. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, 2018.
- [29] Michael Matheny and et al. Using surrogate-based modeling to predict optimal I/O parameters of applications at the extreme scale. In *20th IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, pages 568–575, 2014.
- [30] Robert McLay, Doug James, Si Liu, John Cazes, and William Barth. A user-friendly approach for tuning parallel file operations. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 229–236, 2014.
- [31] Martin Pelikan, David E. Goldberg, and Erick Cantú-Paz. BOA: The Bayesian Optimization Algorithm. In *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation*, 1999.
- [32] S3D-IO code. <https://github.com/wkliao/S3D-IO>.
- [33] Burr Settles. Active Learning Literature Survey. Technical report, University of Wisconsin-Madison, 2009. Computer Sciences TR 1648.
- [34] Huaiming Song, Yanlong Yin, Yong Chen, and Xian-He Sun. Cost-intelligent application-specific data layout optimization for parallel file systems. *Cluster computing*, 16(2):285–298, 2013.
- [35] Richard Vuduc, James W. Demmel, and Jeff A. Birmes. Statistical models for automatic performance tuning. In *Proceedings of the International Conference on Computational Science*, volume 2073 of *LNCs*, pages 117–126, San Francisco, CA, May 2001. Springer.
- [36] R. Clinton Whaley, Antoine Petit, and Jack J. Dongarra. Automated empirical optimizations of software and the ATLAS project. *Parallel Computing*, 27(1-2):3–35, 2001.
- [37] Bing Xie and et al. Predicting output performance of a petascale supercomputer. In *Proceedings of the 26th International Symposium on High-Performance Parallel and Distributed Computing*, 2017.